

Centro Universitário Católica SC

Engenharia de Software — 5ª Fase

Projeto e Arquitetura de Software

Prof. Luiz C. Camargo, PhD

BarberManager

Análise Completa de Arquitetura,
Inspeção de Código e Qualidade de Software

Miguel Angel Balladares Huertas

8 de junho de 2026

Repositório do Projeto

<https://github.com/miguett Huertas/BarberManager>

Sumário

1	Visão Geral do Sistema	1
1.1	Objetivos do Sistema	1
1.2	Stack Tecnológica	1
2	Arquitetura do Sistema — Visão C4	1
2.1	Diagrama de Contexto (Nível 1)	1
2.2	Arquitetura em Camadas	2
2.3	Modelagem da Base de Dados	3
3	Boas Práticas Identificadas e Implementadas	3
3.1	Conexão Per-Request — Thread Safety	3
3.2	Middleware CORS e Auto-Status	4
3.3	Queries Seguras com LEFT JOIN e COALESCE	5
3.4	Segurança — Hash SHA-256 para Passwords	6
3.5	Padrão SPA com Injeção Dinâmica de Frames	6
3.6	Frontend Web — SPA sem Frameworks	7
3.7	Deploy via Tunnel — Sem Servidor Dedicado	7
4	Inspeção de Código — Violações SOLID	7
4.1	Metodologia de Inspeção	7
4.2	Violação 1 — SRP: RotaCriar em API.Agendamentos.pas	7
4.3	Violação 2 — SRP: RotaKPIs em API.Dashboard.pas	8
4.4	Violação 3 — OCP: Array de Meses Hardcoded	8
4.5	Violação 4 — OCP: Validação de Status por Comparação	9
4.6	Violação 5 — DIP: Dependência Direta do FireDAC	9
4.7	Violação 6 — DIP: Hash de Password Acoplada	10
5	Inspeção de Código — Clean Code e DRY	10
5.1	Nomenclatura — 5 Ocorrências	10
5.2	Comentários Problemáticos — 5 Ocorrências	11
5.3	DRY — Duplicação de Lógica	11
6	Métricas de Qualidade	12
6.1	Resumo das Violações	12
6.2	Estado Geral do Projeto	13
7	Roadmap de Melhorias	13
7.1	Prioridade Alta	13

7.2	Prioridade Média	13
7.3	Prioridade Baixa (Evolução Futura)	14
8	Conclusão	14

1 Visão Geral do Sistema

O **BarberManager** é um sistema completo de gestão para barbearias, desenvolvido como projeto acadêmico da disciplina de Programação Web (5ª Fase — Engenharia de Software). O sistema foi concebido para eliminar o atrito dos agendamentos manuais e oferecer controlo financeiro robusto ao proprietário da barbearia.

1.1 Objetivos do Sistema

- **App Mobile/Desktop (Clientes):** Cadastro, autenticação, catálogo de serviços dinâmico, agendamento completo (serviço + barbeiro + horário), meus agendamentos e avaliação pós-serviço.
- **Dashboard Web/Desktop (Administradores):** KPIs financeiros em tempo real, linha do tempo da agenda do dia, gestão completa de serviços, clientes, financeiro e configurações.
- **Acesso Web Público:** Deploy via ngrok com domínio fixo, sem necessidade de servidor dedicado.

1.2 Stack Tecnológica

Camada	Tecnologia / Detalhe
Linguagem	Delphi 12 Athens (Object Pascal)
UI Framework	FireMonkey (FMX) — cross-platform nativo
REST API	Horse Framework — servidor HTTP na porta 9000
Frontend Web	HTML5 / CSS3 / JavaScript puro (SPA single-file)
Base de Dados	Firebird 3.0 (SGBD relacional)
Acesso a Dados	FireDAC — componentes criados 100% por código
Deploy	ngrok Tunnel — URL pública permanente sem servidor
Controlo de Versão	Git + GitHub

Tabela 1: Stack tecnológica completa do BarberManager

2 Arquitetura do Sistema — Visão C4

2.1 Diagrama de Contexto (Nível 1)

O sistema opera com dois tipos de utilizadores e uma base de dados centralizada:

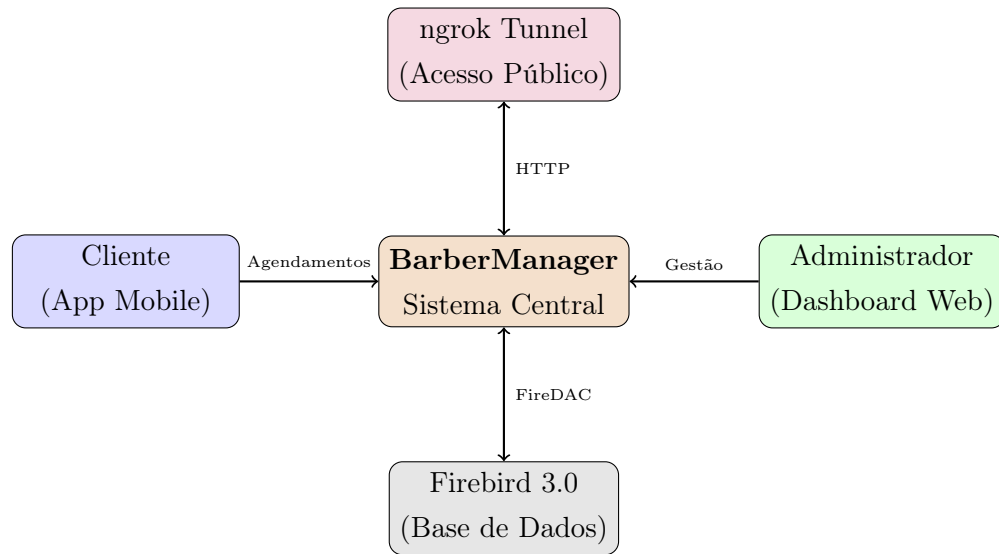


Figura 1: Diagrama de Contexto — BarberManager

2.2 Arquitetura em Camadas

O sistema implementa uma separação em três camadas distintas, alinhada com o padrão MVC:

Camada	Componentes	Responsabilidade
View	View.Principal.pas, View.DashboardAdmin.pas, View.Frame.*.pas, src/Web/index.html	Apresentação e interação com o utilizador
Controller (API)	API.Auth.pas, API.Agendamentos.pas, API.Dashboard.pas, API.Servicos.pas, API.Clientes.pas, API.Avaliacoes.pas, API.Usuarios.pas	Regras de negócio e roteamento HTTP
Model	Model.Conexao.pas, API.Conexao.pas, BARBERMANAGER.FDB	Acesso a dados e persistência

Tabela 2: Separação em camadas do BarberManager

2.3 Modelagem da Base de Dados

O esquema relacional é composto por 7 tabelas, com integridade referencial garantida por chaves estrangeiras:

Esquema principal — tabelas e relações

```
1  -- Utilizadores (clientes e admins)
2  TB_USUARIOS (ID, NOME_COMPLETO, EMAIL UNIQUE, SENHA_HASH, PERFIL,
3              ATIVO, DT_CADASTRO, TELEFONE)
4
5  -- Barbeiros (utilizadores com perfil especial)
6  TB_BARBEIROS (ID, USUARIO_ID FK, CARGO, AVALIACAO_MEDIA, ATIVO)
7
8  -- Catálogo de serviços
9  TB_CATEGORIAS (ID, NOME, COR_TAG)
10 TB_SERVICOS (ID, CATEGORIA_ID FK, NOME, DESCRICAO, PRECO,
11             DURACAO_MIN, BADGE, ATIVO)
12
13 -- Agendamentos (entidade central)
14 TB_AGENDAMENTOS (ID, CLIENTE_ID FK, BARBEIRO_ID FK, SERVICOS_ID FK,
15                  DT_AGENDAMENTO, HR_INICIO, HR_FIM, VALOR_COBRADO,
16                  STATUS, DT_CRIACAO)
17
18 -- Avaliações e configurações
19 TB_AVALIACOES (ID, AGENDAMENTO_ID FK, CLIENTE_ID FK,
20               BARBEIRO_ID FK, NOTA, COMENTARIO, DT_AVALIACAO)
21 TB_CONFIGURACOES (ID, CHAVE UNIQUE, VALOR, DESCRICAO)
```

3 Boas Práticas Identificadas e Implementadas

Esta secção documenta as decisões arquiteturais e padrões de qualidade efetivamente implementados no projeto.

3.1 Conexão Per-Request — Thread Safety

Padrão Implementado: Conexão por Requisição

O Horse é um servidor HTTP multi-thread. Para evitar race conditions, cada handler da API instancia uma conexão FireDAC independente, usa-a e liberta-a no bloco `finally`. Isto garante thread safety sem recorrer a locks ou pools complexos.

```
1 function CreateConnection: TFDConnection;
2 begin
3     Result := TFDConnection.Create(nil);
4     Result.Params.DriverID := 'FB';
5     Result.Params.Database := DATABASE_PATH;
6     Result.Params.UserName := 'SYSDBA';
7     Result.Params.Password := 'masterkey';
8     Result.TxOptions.Isolation := xiReadCommitted;
9     Result.Connected := True;
10 end;
11
12 // Uso nos handlers:
13 procedure RotaListar(Req: THorseRequest; Res: THorseResponse;
14     Next: TProc);
15 var Conn: TFDConnection;
16 begin
17     Conn := CreateConnection;
18     try
19         // ... lógica da rota
20     finally
21         Conn.Free; // sempre libertado
22     end;
23 end;
```

Padrão per-request em API.Conexao.pas

3.2 Middleware CORS e Auto-Status

O servidor Horse implementa dois middlewares globais executados antes de qualquer rota:

```
1 // 1. CORS -- permite acesso de qualquer origem
2 THorse.Use(
3     procedure(Req: THorseRequest; Res: THorseResponse; Next: TProc)
4     begin
5         Res.AddHeader('Access-Control-Allow-Origin', '*');
6         Res.AddHeader('Access-Control-Allow-Methods',
7             'GET, POST, PUT, DELETE, OPTIONS');
8         if Req.Method = 'OPTIONS' then
9             begin
10                 Res.Status(204).Send('');
11                 Exit;
12             end;
```

```

13     Next;
14 end
15 );
16
17 // 2. Auto-status -- atualiza PENDENTE->EM_ANDAMENTO->CONCLUIDO
18 THorse.Use(
19     procedure(Req: THorseRequest; Res: THorseResponse; Next: TProc)
20     begin
21         if Req.Method = 'GET' then
22             API.Agendamentos.AutoAtualizarStatus;
23             Next;
24         end
25     );

```

Middleware globais no BarberManagerAPI.dpr

3.3 Queries Seguras com LEFT JOIN e COALESCE

Padrão Implementado: Proteção contra NULL

Todas as queries que envolvem JOINS utilizam LEFT JOIN em vez de INNER JOIN, e COALESCE para tratar valores NULL. Isto evita Access Violations no FireDAC quando existem registos órfãos ou resultados vazios.

Padrão RDB\$DATABASE para agregados sem linhas

```

1 SELECT
2     COALESCE(Q.FATURAMENTO, 0) AS FATURAMENTO,
3     COALESCE(Q.TOTAL, 0)      AS TOTAL,
4     COALESCE(Q.PENDENTES, 0)  AS PENDENTES
5 FROM RDB$DATABASE
6 LEFT JOIN (
7     SELECT
8         SUM(CASE WHEN STATUS='CONCLUIDO' THEN VALOR_COBRADO ELSE 0 END)
9         AS FATURAMENTO,
9         COUNT(*) AS TOTAL,
10        SUM(CASE WHEN STATUS='PENDENTE' THEN 1 ELSE 0 END) AS PENDENTES
11 FROM TB_AGENDAMENTOS
12 WHERE DT_AGENDAMENTO = :DATA
13 ) Q ON 1=1

```


3.4 Segurança — Hash SHA-256 para Passwords

Padrão Implementado: Armazenamento Seguro de Passwords

As passwords nunca são armazenadas em texto simples. É utilizado SHA-256 em maiúsculas, garantindo que mesmo um acesso direto à base de dados não expõe as credenciais.

```
1 function HashSenha(const Senha: string): string;
2 begin
3     Result := UpperCase(
4         THashSHA2.GetHashString(Senha, THashSHA2.TSHA2Version.SHA256)
5     );
6 end;
```

Função de hashing em API.Auth.pas

3.5 Padrão SPA com Injeção Dinâmica de Frames

O Dashboard Admin implementa o padrão Single Page Application nativamente em FMX: os frames são instanciados e injetados em runtime no contentor `lytAreaPrincipal`, sem recarregar o formulário.

```
1 procedure TFrmDashboardAdmin.rectMenuServicosClick(Sender:
2     TObject);
3 var FrameServicos: TFrameServicos;
4 begin
5     // 1. Limpar frames existentes
6     for I := lytAreaPrincipal.ControlsCount - 1 downto 0 do
7         if lytAreaPrincipal.Controls[I] is TFrame then
8             lytAreaPrincipal.Controls[I].DisposeOf;
9
10    // 2. Esconder dashboard principal
11    scrollDashboard.Visible := False;
12
13    // 3. Criar e injetar novo frame
14    FrameServicos := TFrameServicos.Create(Self);
15    FrameServicos.Parent := lytAreaPrincipal;
16    FrameServicos.Align := TAlignLayout.Client;
17 end;
```

Injeção de frames — View.DashboardAdmin.pas

3.6 Frontend Web — SPA sem Frameworks

O frontend web foi desenvolvido em HTML/CSS/JS puro, sem dependência de frameworks externos (React, Vue, Angular). Isto elimina complexidade de build, reduz o tamanho do bundle e facilita a manutenção. O mesmo ficheiro `index.html` serve toda a aplicação — login, área de cliente e painel administrativo — gerido por estado JavaScript.

3.7 Deploy via Tunnel — Sem Servidor Dedicado

O deploy público é realizado via **ngrok**, expondo o servidor Horse local (`localhost:9000`) com um domínio fixo permanente (`humid-boots-posted.ngrok-free.dev`), sem necessidade de:

- Servidor dedicado ou VPS
- IP fixo ou configuração de router
- Certificado SSL manual (HTTPS automático)

4 Inspeção de Código — Violações SOLID

4.1 Metodologia de Inspeção

A inspeção foi realizada de forma estática e manual sobre os ficheiros principais da API Horse (`API.Agendamentos.pas`, `API.Dashboard.pas`, `API.Auth.pas`, `API.Servicos.pas`), equiparando os módulos procedurais do Object Pascal a classes de objetos para análise dos princípios SOLID.

4.2 Violação 1 — SRP: RotaCriar em API.Agendamentos.pas

Violação: Single Responsibility Principle

A função `RotaCriar` acumula três responsabilidades distintas: parsing do JSON recebido (camada HTTP), cálculo de datas e horários (regra de negócio) e execução do INSERT na base de dados (persistência). Qualquer alteração em qualquer uma destas camadas obriga a modificar a mesma função.

```
1 procedure RotaCriar(Req: THorseRequest; Res: THorseResponse; Next:
   TProc);
2 begin
3   // Responsabilidade 1: Parse HTTP/JSON
4   Body := TJSONObject.ParseJSONValue(Req.Body) as TJSONObject;
```

```
5
6 // Responsabilidade 2: Regra de negócio (cálculo de datas)
7 DtAgen := EncodeDate(StrToIntDef(DataParts[0], 2024),
8                       StrToIntDef(DataParts[1], 1),
9                       StrToIntDef(DataParts[2], 1));
10 HrInicio := StrToTimeDef(HoraStr + ':00', 0);
11 HrFim := IncMinute(HrInicio, DuracaoMin);
12
13 // Responsabilidade 3: Persistência (SQL INSERT)
14 Query.SQL.Text := 'INSERT INTO TB_AGENDAMENTOS ...';
15 Query.ExecSQL;
16 end;
```

Código atual — múltiplas responsabilidades

Refatoração sugerida: Extrair a lógica de negócio para um `AgendamentoService.pas` e a persistência para um `AgendamentoRepository.pas`, deixando a rota apenas responsável por orquestrar a chamada e formatar a resposta HTTP.

4.3 Violação 2 — SRP: RotaKPIs em API.Dashboard.pas

Violação: Single Responsibility Principle

O handler da rota HTTP `RotaKPIs` constrói diretamente queries SQL complexas com múltiplos `COALESCE`, `SUM` e `CASE WHEN` para extração de métricas financeiras. O controlador HTTP não deve conhecer a estrutura da base de dados.

Refatoração sugerida: Criar um `DashboardService.pas` que encapsule toda a lógica de cálculo de KPIs, devolvendo um objeto de dados tipado ao handler, que apenas serializa a resposta JSON.

4.4 Violação 3 — OCP: Array de Meses Hardcoded

Violação: Open-Closed Principle

O array `MESES`: `array[1..12] of string` com os nomes dos meses em português está rigidamente codificado. Se o sistema precisar de suportar outro idioma (internacionalização), será necessário modificar o código-fonte.

```
1 const
2   MESES: array[1..12] of string = (
3     'Janeiro', 'Fevereiro', 'Março', 'Abril', 'Maio', 'Junho',
```

```
4      'Julho', 'Agosto', 'Setembro', 'Outubro', 'Novembro',  
5      'Dezembro'  
6  );
```

Código atual — fechado para extensão

Refatoração sugerida: Utilizar `SysUtils.FormatSettings` para obter os nomes dos meses de forma dinâmica a partir das configurações de localidade do sistema operativo.

4.5 Violação 4 — OCP: Validação de Status por Comparação

Violação: Open-Closed Principle

A validação do campo `status` utiliza comparações explícitas com strings. Sempre que um novo estado for adicionado (ex: `AGUARDANDO_PAGAMENTO`), esta condicional precisará de ser modificada — violando o princípio aberto/fechado.

```
1  if not (NovoStatus = 'PENDENTE') and  
2      not (NovoStatus = 'EM_ANDAMENTO') and  
3      not (NovoStatus = 'CONCLUIDO') and  
4      not (NovoStatus = 'CANCELADO') then  
5  begin  
6      Res.Status(400).Send('Status inválido');  
7      Exit;  
8  end;
```

Código atual — lista de estados hardcoded

Refatoração sugerida: Definir um tipo enumerado `TStatusAgendamento` e uma função de conversão, ou validar contra uma lista gerida em `TB_CONFIGURACOES`.

4.6 Violação 5 — DIP: Dependência Direta do FireDAC

Violação: Dependency Inversion Principle

Os módulos de alto nível (rotas da API) dependem diretamente dos módulos de baixo nível de persistência (`TFDConnection`, `TFDQuery`). Isto impossibilita a substituição da base de dados ou a realização de testes unitários com mocks.

```
1  var  
2      Conn: TFDConnection;  
3      Query: TFDQuery;  
4  begin  
5      Conn := CreateConnection; // acoplado ao FireDAC
```

```

6 Query := TFDQuery.Create(nil);
7 Query.Connection := Conn;
8 // ...

```

Código atual — acoplamento direto

Refatoração sugerida: Definir interfaces `IConnection` e `IQuery` que abstraíam o acesso a dados, permitindo injetar implementações alternativas (SQLite para testes, PostgreSQL no futuro).

4.7 Violação 6 — DIP: Hash de Password Acoplada

Violação: Dependency Inversion Principle

A função `HashSenha` está duplicada em `API.Auth.pas` e `API.Usuarios.pas`, e ambas estão fortemente acopladas à implementação específica `THashSHA2`. Uma migração para `Bcrypt` exigiria modificações em múltiplos ficheiros.

Refatoração sugerida: Criar uma interface `IHashService` implementada por `SHA256HashService` e injetá-la nas units que precisam de hashing, centralizando a lógica numa única unidade partilhada (`Utils.Seguranca.pas`).

5 Inspeção de Código — Clean Code e DRY

5.1 Nomenclatura — 5 Ocorrências

Ocorrência	Código Atual	Refatoração
Abreviações ilegíveis	<code>FatConc</code> , <code>FatTotal</code>	<code>FaturamentoConcluido</code> , <code>FaturamentoTotal</code>
Nomes genéricos	<code>Q1</code> , <code>Q2</code> : <code>TFDQuery</code>	<code>QueryTotaisMensais</code> , <code>QueryValoresPorDia</code>
Abreviações confusas	<code>DtAgen</code> : <code>TDate</code>	<code>DataDoAgendamento</code> : <code>TDate</code>
Notação húngara	<code>QCheck</code> , <code>QInsert</code> , <code>QUpdate</code>	<code>ConsultaVerificacao</code> , <code>ConsultaInsercao</code>
Termos genéricos	<code>Item</code> : <code>TJSONObject</code>	<code>JsonAgendamento</code> : <code>TJSONObject</code>

Tabela 3: Ocorrências de nomenclatura inadequada

5.2 Comentários Problemáticos — 5 Ocorrências

1. **Comentário a suprir falta de abstração:** `// Parse 'YYYY-MM-DD' -> TDate` via `EncodeDate` — deveria ser extraído para `ConverterDataISO(DataString: string): TDate`.
2. **Justificação de solução problemática:** `// append ':00' so StrToTimeDef` never gets a partial string — o comentário admite uma gambiarra. A solução correta seria um parser de hora robusto.
3. **Comentário redundante:** `// FormatDateTime 'hh:nn:ss' = locale-independent` HH:MM:SS — o próprio código já é auto-explicativo.
4. **Exceção silenciada:** O bloco `except // Silent - status update is best-effort` oculta erros de base de dados. Deveria registrar o erro num ficheiro de log.
5. **Writeln de debug em produção:** `Writeln('Database driver loaded.')` e similares são resquícios da fase de desenvolvimento que poluem o output em produção.

5.3 DRY — Duplicação de Lógica

Violação: Don't Repeat Yourself

A função `HashSenha` está integralmente duplicada em `API.Auth.pas` e `API.Usuarios.pas`. Qualquer alteração no algoritmo de hashing exige modificação em dois lugares, com risco de inconsistência.

Solução imediata: Criar `Utils.Seguranca.pas` com a função centralizada:

```

1 unit Utils.Seguranca;
2
3 interface
4
5 function HashSenha(const Senha: string): string;
6
7 implementation
8
9 uses System.Hash;
10
11 function HashSenha(const Senha: string): string;
12 begin
13     Result := UpperCase(

```

```

14     THashSHA2.GetHashString(Senha, THashSHA2.TSHA2Version.SHA256)
15 );
16 end;
17
18 end.

```

Utils.Seguranca.pas — função centralizada

6 Métricas de Qualidade

6.1 Resumo das Violações

Categoria	Descrição	Qtd.	Severidade
SOLID — SRP	Múltiplas responsabilidades por função	2	Alta
SOLID — OCP	Código fechado para extensão	2	Média
SOLID — DIP	Acoplamento a implementações concretas	2	Alta
Clean Code — Nomes	Abreviações e nomes genéricos	5	Baixa
Clean Code — Comentários	Redundantes, enganadores, debug	5	Baixa
DRY	Duplicação de lógica crítica	1	Média
Total		17	

Tabela 4: Resumo das violações identificadas

6.2 Estado Geral do Projeto

Aspeto Avaliado	Estado
Separação View / Controller / Model	Implementado
Segurança de passwords (SHA-256)	Implementado
Thread safety (per-request connection)	Implementado
Proteção contra NULL (COALESCE)	Implementado
Middleware CORS e auto-status	Implementado
Deploy público com HTTPS	Implementado
Testes unitários	Ausente
Injeção de dependências	Ausente
Camada de serviço (Service Layer)	Ausente
Logging estruturado	Ausente
Paginação nas listagens	Ausente

Tabela 5: Estado geral do projeto

7 Roadmap de Melhorias

7.1 Prioridade Alta

1. **Centralizar HashSenha:** criar `Utils.Seguranca.pas` e remover duplicação — impacto imediato na manutenibilidade.
2. **Logging estruturado:** substituir os blocos `except // Silent` por escrita em ficheiro de log com timestamp, nível e mensagem.
3. **Nomenclatura:** renomear variáveis abreviadas nas units de API — sem impacto funcional, melhoria imediata de legibilidade.

7.2 Prioridade Média

1. **Service Layer:** extrair regras de negócio das rotas HTTP para unidades de serviço dedicadas (`AgendamentoService.pas`, `DashboardService.pas`).
2. **Enumerados de status:** substituir strings hardcoded por tipo enumerado `TStatusAgendamento`.
3. **Paginação:** implementar parâmetros `page` e `limit` nas listagens de agendamentos e clientes.

7.3 Prioridade Baixa (Evolução Futura)

1. **Testes unitários:** implementar framework de testes (DUnitX) com mocks para as camadas de serviço.
2. **Internacionalização:** substituir arrays de meses por `FormatSettings` dinâmico.
3. **Containerização:** migrar a base de dados Firebird para um container Docker para facilitar o deploy.
4. **URL permanente:** migrar do ngrok free para um servidor dedicado (VPS) com domínio próprio e certificado SSL permanente.

8 Conclusão

O BarberManager demonstra um nível de maturidade técnica acima do esperado para um projeto acadêmico de 5ª fase. A decisão de utilizar uma stack não convencional (Delphi FMX + Horse REST + HTML/JS) e entregar um sistema funcionalmente completo, com deploy público, revela competência técnica e capacidade de resolução de problemas reais.

As violações identificadas — 6 de SOLID, 10 de Clean Code e 1 de DRY — são características de um sistema em fase de crescimento rápido, onde a entrega de funcionalidades foi priorizada sobre a refatoração. Nenhuma das violações compromete o funcionamento do sistema; todas são oportunidades de evolução que tornariam o código mais testável, extensível e manutenível a longo prazo.

O projeto cumpre integralmente os requisitos da disciplina e demonstra domínio de conceitos fundamentais de arquitetura de software, incluindo separação de responsabilidades, segurança de dados, padrões de acesso a dados thread-safe e deploy em ambiente de produção.

Miguel Angel Balladares Huertas
Engenharia de Software — 5ª Fase
Centro Universitário Católica SC
8 de junho de 2026